

Psych 250a — Week 0 Assignment. Introduction to R

Seongmin Park

September 30, 2026

Contents

Question 1: Import and Summarize the Data	1
Question 2: Decisions About the Depression Sum Score	4
Question 3: AI Prompt for Sum Score	4
Question 4: Audit the Sum Score Code	5
Question 5: AI Prompt for Verification Checks	5
Question 6: Explain the Verification Code	6
Question 7: Recode Depression into a Binary Variable	6
Question 8: Convert Binary Variables to Factors	7
Question 9: AI Review of Recoding and Factor Code	7
Question 10: Chat Log	9

Question 1: Import and Summarize the Data

```
library(summarytools)

filepath <-
  ↪ "https://raw.githubusercontent.com/craigenders/psych250a/main/data/assignment0.csv"
assign0 <- read.csv(filepath, stringsAsFactors = T)

dfSummary(assign0)
```

Data Frame Summary
assign0
Dimensions: 275 x 18

Duplicates: 0

No	Variable	Stats / Values	Freqs (% of Valid)	Graph
↪	Valid	Missing		

1	ID	Mean (sd) : 138 (79.5)	275 distinct values	: : : : : : : : : : 275
↪	0			
	[integer]	min < med < max:	(Integer sequence)	: : : : : : : : : :
	↪ (100.0%)	(0.0%)		
		1 < 138 < 275		: : : : : : : : : :
		IQR (CV) : 137 (0.6)		: : : : : : : : : :
				: : : : : : : : : :
2	Female	Min : 0	0 : 179 (65.1%)	IIIIIIIIIIIIIIIIII 275
↪	0			
	[integer]	Mean : 0.3	1 : 96 (34.9%)	IIIIIII
	↪ (100.0%)	(0.0%)		
		Max : 1		
3	Age	Mean (sd) : 44.5 (11)	51 distinct values	: 275
↪	0			
	[integer]	min < med < max:		. : :
	↪ (100.0%)	(0.0%)		
		19 < 45 < 78		: : : :
		IQR (CV) : 16 (0.2)		: : : : : :
				. : : : : : : . .
4	Treatment	Min : 0	0 : 142 (51.6%)	IIIIIIIIIIIIIIIIII 275
↪	0			
	[integer]	Mean : 0.5	1 : 133 (48.4%)	IIIIIIIIIIIIIIIIII
	↪ (100.0%)	(0.0%)		
		Max : 1		
5	Exercise	Mean (sd) : 6 (3.7)	139 distinct values	: . 275
↪	0			
	[numeric]	min < med < max:		. : : :
	↪ (100.0%)	(0.0%)		
		0 < 5.4 < 16.2		: : : : .
		IQR (CV) : 5 (0.6)		: : : : : . .
				: : : : : : :
6	Depress1	Mean (sd) : 2.1 (1)	1 : 94 (34.2%)	IIIIIII 275
↪	0			
	[integer]	min < med < max:	2 : 97 (35.3%)	IIIIIIII
	↪ (100.0%)	(0.0%)		
		1 < 2 < 4	3 : 57 (20.7%)	IIII
		IQR (CV) : 2 (0.5)	4 : 27 (9.8%)	I
7	Depress2	Mean (sd) : 2.6 (1)	1 : 36 (13.1%)	II 275
↪	0			
	[integer]	min < med < max:	2 : 99 (36.0%)	IIIIIIII
	↪ (100.0%)	(0.0%)		

		1 < 3 < 4	3 : 76 (27.6%)	IIIII	
		IQR (CV) : 1 (0.4)	4 : 64 (23.3%)	IIII	
8	Depress3	Mean (sd) : 2 (1.1)	1 : 112 (40.7%)	IIIIIIII	275
	↪ 0				
	[integer]	min < med < max:	2 : 79 (28.7%)	IIIII	
	↪ (100.0%)	(0.0%)			
		1 < 2 < 4	3 : 47 (17.1%)	III	
		IQR (CV) : 2 (0.5)	4 : 37 (13.5%)	II	
9	Depress4	Mean (sd) : 2.3 (1.1)	1 : 73 (27.5%)	IIIII	265
	↪ 10				
	[integer]	min < med < max:	2 : 91 (34.3%)	IIIIII	
	↪ (96.4%)	(3.6%)			
		1 < 2 < 4	3 : 52 (19.6%)	III	
		IQR (CV) : 2 (0.5)	4 : 49 (18.5%)	III	
10	Depress5	Mean (sd) : 2.1 (1)	1 : 90 (34.0%)	IIIIII	265
	↪ 10				
	[integer]	min < med < max:	2 : 92 (34.7%)	IIIIII	
	↪ (96.4%)	(3.6%)			
		1 < 2 < 4	3 : 52 (19.6%)	III	
		IQR (CV) : 2 (0.5)	4 : 31 (11.7%)	II	
11	Depress6	Mean (sd) : 1.9 (1.1)	1 : 131 (49.4%)	IIIIIIIIII	265
	↪ 10				
	[integer]	min < med < max:	2 : 68 (25.7%)	IIIII	
	↪ (96.4%)	(3.6%)			
		1 < 2 < 4	3 : 30 (11.3%)	II	
		IQR (CV) : 1 (0.6)	4 : 36 (13.6%)	II	
12	Depress7	Mean (sd) : 1.6 (0.9)	1 : 164 (61.9%)	IIIIIIIIIIII	265
	↪ 10				
	[integer]	min < med < max:	2 : 53 (20.0%)	IIII	
	↪ (96.4%)	(3.6%)			
		1 < 1 < 4	3 : 29 (10.9%)	II	
		IQR (CV) : 1 (0.6)	4 : 19 (7.2%)	I	
13	Interf1	Mean (sd) : 4.1 (1.7)	1 : 31 (11.3%)	II	275
	↪ 0				
	[integer]	min < med < max:	2 : 25 (9.1%)	I	
	↪ (100.0%)	(0.0%)			
		1 < 4 < 6	3 : 37 (13.5%)	II	
		IQR (CV) : 3 (0.4)	4 : 51 (18.5%)	III	
			5 : 50 (18.2%)	III	
			6 : 81 (29.5%)	IIII	
14	Interf2	Mean (sd) : 4.2 (1.8)	1 : 38 (13.8%)	II	275
	↪ 0				
	[integer]	min < med < max:	2 : 24 (8.7%)	I	
	↪ (100.0%)	(0.0%)			
		1 < 5 < 6	3 : 37 (13.5%)	II	
		IQR (CV) : 3 (0.4)	4 : 33 (12.0%)	II	
			5 : 38 (13.8%)	II	
			6 : 105 (38.2%)	IIIIII	

```

15  Interf3      Mean (sd) : 4.2 (1.7)    1 : 26 ( 9.5%)    I                275
    ↪ 0
      [integer]  min < med < max:        2 : 31 (11.3%)    II
      ↪ (100.0%) (0.0%)
          1 < 5 < 6          3 : 28 (10.2%)    II
          IQR (CV) : 3 (0.4)  4 : 51 (18.5%)    III
                                5 : 57 (20.7%)    IIII
                                6 : 82 (29.8%)    IIIII

16  Interf4      Mean (sd) : 4.4 (1.7)    1 : 24 ( 8.7%)    I                275
    ↪ 0
      [integer]  min < med < max:        2 : 21 ( 7.6%)    I
      ↪ (100.0%) (0.0%)
          1 < 5 < 6          3 : 36 (13.1%)    II
          IQR (CV) : 3 (0.4)  4 : 36 (13.1%)    II
                                5 : 60 (21.8%)    IIII
                                6 : 98 (35.6%)    IIIIIII

17  Interf5      Mean (sd) : 2.8 (1.9)    1 : 115 (41.8%)   IIIIIIIII        275
    ↪ 0
      [integer]  min < med < max:        2 : 36 (13.1%)    II
      ↪ (100.0%) (0.0%)
          1 < 2 < 6          3 : 21 ( 7.6%)    I
          IQR (CV) : 4 (0.7)  4 : 25 ( 9.1%)    I
                                5 : 42 (15.3%)    III
                                6 : 36 (13.1%)    II

18  Interf6      Mean (sd) : 4.3 (1.6)    1 : 24 ( 8.7%)    I                275
    ↪ 0
      [integer]  min < med < max:        2 : 31 (11.3%)    II
      ↪ (100.0%) (0.0%)
          1 < 5 < 6          3 : 23 ( 8.4%)    I
          IQR (CV) : 3 (0.4)  4 : 40 (14.5%)    II
                                5 : 75 (27.3%)    IIIII
                                6 : 82 (29.8%)    IIIII

```

```

    ↪ -----

```

Question 2: Decisions About the Depression Sum Score

Your explanation of which recommendations dictate how the sum score is computed, and why:

According to the message I got, ‘3. Decide Between Sum Scores and Mean Scores’ and ‘4. Handle Missing Data’ sections told me how to compute the sum score. We didn’t learn a concept of reliability yet (so did psych package, which is mentioned in “1. verify that the items measure the same construct) and items related to depression do not need to be reverse-coded (which is mentioned in”2. Reverse-coded negatively worded items). So I directly moved down to the third and fourth section, which provides the code with considering the missing data.

Question 3: AI Prompt for Sum Score

AI’s code:

```
assign0$Depression <- rowSums(
assign0[, c("Depress1",
"Depress2",
"Depress3",
"Depress4",
"Depress5",
"Depress6",
"Depress7")]
)
```

Question 4: Audit the Sum Score Code

a. Which part of the code selects `Depress1` to `Depress7`?

```
c("Depress1", "Depress2", "Depress3", "Depress4", "Depress5", "Depress6", "Depress7")
```

b. Which part sums the seven variables separately for each participant?

```
rowSums( assign0[, c()] )
```

c. Which part produces a missing sum score (NA) if any of the seven items is missing?

Default setting of `rowSums()` function is as follows: `rowSums (x, na.rm = FALSE, dims = 1)` `na.rm = FALSE` means 'do not remove the NA values from the dataset'.

d. Which part adds the sum score named `Depression` to the data frame?

```
assign0$Depression <- rowSums()
```

Question 5: AI Prompt for Verification Checks

AI's code:

```
# Verify that no score exists when any item is missing

sum(
rowSums(is.na(assign0[, c("Depress1", "Depress2", "Depress3",
"Depress4", "Depress5", "Depress6",
"Depress7"]))) > 0 &
!is.na(assign0$Depression)
)
```

```
[1] 0
```

```
# Verify that the number of complete cases equals the number of non-missing scores
```

```
sum(complete.cases(
assign0[, c("Depress1", "Depress2", "Depress3",
"Depress4", "Depress5", "Depress6",
"Depress7")])
)) == sum(!is.na(assign0$Depression))
```

```
[1] TRUE
```

Question 6: Explain the Verification Code

Codebook paragraph for Check 1:

First check: ‘Verify that no score exists when any time is missing’

At first, The code `is.na(assign0[, c("Depress1", "Depress2", "Depress3", "Depress4", "Depress5", "Depress6", "Depress7")])` provides information about ‘If there is a missing value in specific variables (items), then the output will be ‘TRUE’, which is 1. On the other hand, if there is no missing value, the output will be ‘FALSE’, which is 0. Then, the code `rowSums() > 0` computes the number of missing values in each row and tests whether the value is larger than 0. For example, if there is an individual who didn’t respond to 4 items (i.e., 4 ‘TRUE’s), then the result value would be 4, which is larger than 0. And thus, the result of the code is ‘TRUE’. Secondly, the code `is.na(assign0$Depression)` provides information about ‘If there is a missing value in ‘Depression’, which is a summed score that we just made before, then the output will be ‘TRUE’, which is 1. If there is no missing value, the output will be ‘FALSE’, which is 0. **HOWEVER**, you could check the `!` symbol, which reverses the role of the function. Thus, `!is.na()` provides information about ‘If there is **NOT** a missing value in ‘Depression’, then the output will be ‘TRUE’. Finally, the logical symbol `&` conduct a logical test between the two values we got. R coerces nonzero values to ‘TRUE’ and zero to ‘FALSE’. Ideally, the pair of two values must be different (e.g., TRUE and FALSE or FALSE and TRUE), which should represent all the result values as ‘FALSE’, and the `sum()` of these values should be **ZERO**. Otherwise, if there is at least one missing in each variable (which is TRUE) but if the ‘Depression’ still has a value (which is also TRUE), the result will be ‘TRUE’, and the result will exhibit the value larger than ZERO, indicating the sum score was computed incorrectly.

Codebook paragraph for Check 2:

Second check: ‘Verify that the number of complete cases equals the number of non-missing scores’

At first, the function `complete.cases()` provides information whether the cases are complete or not. `complete.cases( assign0[, c("Depress1", "Depress2", "Depress3", "Depress4", "Depress5", "Depress6", "Depress7")])` will exhibit ‘TRUE’ if the case is complete, or ‘FALSE’ if the case is incomplete, which includes at least one missing value. Secondly, `!is.na(assign0$Depression)` is same with the first check, which will exhibit ‘TRUE’ if there is NOT a missing value in ‘Depression’, or ‘FALSE’ if there is NA in ‘Depression’. Finally, the logical symbol `==` conduct a logical test between sums of these two values. Ideally, the result should be **TRUE**, which indicates the number of complete cases equals to the number of non-missing value in ‘Depression’. Otherwise, if there is a mismatch, the result will represent FALSE, indicating the sum score was computed incorrectly.

Question 7: Recode Depression into a Binary Variable

Your code:

```
# create ClinicalDep: 1 if Depression is 17 or higher, otherwise 0
assign0$ClinicalDep <- ifelse(assign0$Depression >= 17, 1, 0)
```

Question 8: Convert Binary Variables to Factors

Your code:

```
assign0$ClinicalDep <- factor(assign0$ClinicalDep,
                              levels = c(0, 1),
                              labels = c("Subclinical", "Clinical"))

assign0$Female <- factor(assign0$Female,
                         levels = c(0, 1),
                         labels = c("Male", "Female"))

assign0$Treatment <- factor(assign0$Treatment,
                            levels = c(0, 1),
                            labels = c("Control", "Treatment"))
```

Question 9: AI Review of Recoding and Factor Code

AI's verification code:

```
# Recode

sum(
  assign0$Depression >= 17 &
  assign0$ClinicalDep == "Subclinical",
  na.rm = TRUE
)
```

[1] 0

```
sum(
  assign0$Depression < 17 &
  assign0$ClinicalDep == "Clinical",
  na.rm = TRUE
)
```

[1] 0

```
with(assign0,
      table(
        ClinicalDep,
        Depression >= 17,
        useNA = "always"
      ))
```

ClinicalDep	FALSE	TRUE	<NA>
Subclinical	177	0	0
Clinical	0	88	0
<NA>	0	0	10

```
sum(is.na(assign0$Depression)) ==  
sum(is.na(assign0$ClinicalDep))
```

```
[1] TRUE
```

```
# Factor code
```

```
table(assign0$ClinicalDep, useNA = "always")
```

Subclinical	Clinical	<NA>
177	88	10

```
table(assign0$Female, useNA = "always")
```

Male	Female	<NA>
179	96	0

```
table(assign0$Treatment, useNA = "always")
```

Control	Treatment	<NA>
142	133	0

```
levels(assign0$ClinicalDep)
```

```
[1] "Subclinical" "Clinical"
```

```
levels(assign0$Female)
```

```
[1] "Male" "Female"
```

```
levels(assign0$Treatment)
```

```
[1] "Control" "Treatment"
```

Your statement about what the verification shows:

- First 2 codes verify whether the cutoff is well-applied. Since there is no mismatch (which represent a value 0), this verifies that the cutoff is well-applied.

- Next code generates the table, which not only summarize the frequency but also verifies whether the labels are well-defined.
- Next code verifies whether missing values are handled appropriately. Since the result exhibits TRUE, we can verify that the missing values (NAs) are maintained.
- Regarding factor code, 3 table codes represent the frequency tables, which verify whether the labels are well-defined.
- Following three codes also confirm that the factor labels are applied appropriately.

Question 10: Chat Log

Work
Park

Auto

You said:

What checks can I perform in R to verify that the sum score was computed correctly?

Copilot said:

A good practice is to perform a few sanity checks that verify both the summation itself
→ and the missing-data rule.

1. Check a few rows manually

Inspect the original items alongside the computed score.

```
R
head(
  assign0[, c("Depress1", "Depress2", "Depress3",
             "Depress4", "Depress5", "Depress6",
             "Depress7", "Depression")],
  10
)
```

You can then manually verify a few rows:

Row with complete data: does Depression equal the sum of the 7 items?

Row with at least one NA: is Depression also NA?

2. Verify that no score exists when an item is missing

This check should return 0.

```
R
sum(
  rowSums(is.na(assign0[, c("Depress1", "Depress2", "Depress3",
                           "Depress4", "Depress5", "Depress6",
                           "Depress7")])) > 0 &
  !is.na(assign0$Depression)
)
```

Interpretation:

0 = success

Any value > 0 = some rows received a score despite missing item responses

3. Verify that all complete cases received a score

This check should also return 0.

```
R
sum(
  rowSums(is.na(assign0[, c("Depress1", "Depress2", "Depress3",
                           "Depress4", "Depress5", "Depress6",
                           "Depress7")])) == 0 &
  is.na(assign0$Depression)
)
```

Interpretation:

0 = success

Any value > 0 = some complete cases did not receive a score

4. Recompute the sum independently and compare

A very useful validation.

```
R
check_sum <- rowSums(
  assign0[, c("Depress1", "Depress2", "Depress3",
              "Depress4", "Depress5", "Depress6",
              "Depress7")]
)
```

```
all.equal(assign0$Depression, check_sum)
```

Expected result:

```
R
[1] TRUE
```

5. Examine the possible score range

Suppose each item is scored from 1 to 5. Then the valid sum score range should be 7 to

↪ 35.

```
R
summary(assign0$Depression)
or
```

```
R
range(assign0$Depression, na.rm = TRUE)
Look for values outside the theoretically possible range.
```

6. Count complete cases and non-missing scores
The numbers should match exactly.

```
R
sum(complete.cases(
  assign0[, c("Depress1", "Depress2", "Depress3",
             "Depress4", "Depress5", "Depress6",
             "Depress7")]
))
```

```
R
sum(!is.na(assign0$Depression))
These two counts should be identical.
```

One-line diagnostic
This compact test simultaneously checks your scoring rule:

```
R
table(
  Complete = complete.cases(assign0[, c("Depress1", "Depress2", "Depress3",
                                       "Depress4", "Depress5", "Depress6",
                                       "Depress7")]),
  ScorePresent = !is.na(assign0$Depression)
)
The ideal output is:
```

```
R
      ScorePresent
Complete FALSE TRUE
```

```
FALSE   xxx   0
TRUE    0   yyy
```

The two off-diagonal cells should be 0. If they are, your Depression variable follows the
 ↳ rule "sum only when all 7 items are observed" exactly.

You said:

I think "2. Verify that no score exists when an item is missing" and "6. Count complete
 ↳ cases and non-missing scores" options are good for me. Based on the information I
 ↳ provided before:

Name of data frame: assign0

Variables (colnames) for summation: Depress1, Depress2, Depress3, Depress4, Depress5,
 ↳ Depress6, Depress7

Name of a new sum score variable in 'assign0' data frame: Depression

Could you generate (or reexamine) the R code of these two checks?

Copilot said:

Yes. Given your variables and scoring rule, here are streamlined versions of those two
 ↳ checks.

1. Verify that no score exists when any item is missing

This should return 0 if the Depression variable was computed correctly.

R

```
sum(
  rowSums(is.na(assign0[, c("Depress1", "Depress2", "Depress3",
                           "Depress4", "Depress5", "Depress6",
                           "Depress7")])) > 0 &
  !is.na(assign0$Depression)
)
```

Interpretation:

0 = every respondent with at least one missing item has Depression = NA

> 0 = some respondents received a score despite missing one or more items

2. Verify that the number of complete cases equals the number of non-missing scores

These two numbers should be identical.

R

```
sum(complete.cases(
  assign0[, c("Depress1", "Depress2", "Depress3",
              "Depress4", "Depress5", "Depress6",
              "Depress7")])
```

))

R
sum(!is.na(assign0\$Depression))
To test this directly, use:

R
sum(complete.cases(
 assign0[, c("Depress1", "Depress2", "Depress3",
 "Depress4", "Depress5", "Depress6",
 "Depress7")]
)) == sum(!is.na(assign0\$Depression))
Interpretation:

TRUE = the counts match exactly
FALSE = some complete cases are missing a score, or some incomplete cases received a
 ↪ score
Optional: Single diagnostic table
I often prefer this because it immediately shows whether there are any mismatches:

R
table(
 CompleteCase = complete.cases(
 assign0[, c("Depress1", "Depress2", "Depress3",
 "Depress4", "Depress5", "Depress6",
 "Depress7")]
),
 DepressionPresent = !is.na(assign0\$Depression)
)
The ideal output is:

R
 DepressionPresent
CompleteCase FALSE TRUE
 FALSE ... 0
 TRUE 0 ...

The two off-diagonal cells should be 0. If they are, your Depression score follows the
 ↪ rule "compute the sum only when all 7 items are present."

You said:

From now on, I'd like to request you to verify whether my code will work accurately or
→ not. I wrote two different codes:

1. Recode 'Depression' variable, which is a sum score we calculated before, into binary
→ variable named as "ClinicalDep". If the value of 'Depression' is equal to 17 or
→ higher, recode this as 1. Otherwise, if the value of 'Depression' is below 17, recode
→ this as 0. If the value of "Depression" is missing, just keep the value as NA.

My code is as follows:

```
assign0$ClinicalDep <- ifelse(assign0$Depression >= 17, 1, 0)
```

2. Convert Binary Variables to Factors. Not only "ClinicalDep" but also "Female" and
→ "Treatment" variables in "assign0" data frame. levels and labels are as follows:

```
variable "ClinicalDep": 0 = "Subclinical", 1 = "Clinical"  
variable "Female": 0 = "Male", 1 = "Female"  
variable "Treatment": 0 = "Control", 1 = "Treatment"
```

My code is as follows:

```
assign0$ClinicalDep <- factor(assign0$ClinicalDep,  
                             levels = c(0, 1),  
                             labels = c("Subclinical", "Clinical"))
```

```
assign0$Female <- factor(assign0$Female,  
                        levels = c(0, 1),  
                        labels = c("Male", "Female"))
```

```
assign0$Treatment <- factor(assign0$Treatment,  
                           levels = c(0, 1),  
                           labels = c("Control", "Treatment"))
```

And I'd like to request you to provide me some R code to produce some tables or outputs
→ for verification, whether the cutoff was applied correctly and the intended labels
→ were assigned.

Copilot said:

Both of your coding approaches look correct. I'll review each one carefully and then
→ provide verification code.

1. Creating ClinicalDep

Your code:

```
R
assign0$ClinicalDep <- ifelse(assign0$Depression >= 17, 1, 0)
Will it work correctly?
Yes.
```

An important feature of `ifelse()` is that when the test condition is NA, the result is
→ also NA.

For example:

```
R
ifelse(20 >= 17, 1, 0)
# 1
```

```
ifelse(15 >= 17, 1, 0)
# 0
```

```
ifelse(NA >= 17, 1, 0)
# NA
```

Therefore your code implements exactly the requested rule:

```
Depression >= 17 → 1
Depression < 17 → 0
Depression = NA → NA
No changes are necessary.
```

2. Converting Variables to Factors

Your code:

```
R
assign0$ClinicalDep <- factor(assign0$ClinicalDep,
                             levels = c(0, 1),
                             labels = c("Subclinical", "Clinical"))
```

```
assign0$Female <- factor(assign0$Female,
                         levels = c(0, 1),
                         labels = c("Male", "Female"))
```

```
assign0$Treatment <- factor(assign0$Treatment,
                            levels = c(0, 1),
                            labels = c("Control", "Treatment"))
```

```
Will it work correctly?
Yes.
```

This correctly maps:

ClinicalDep

```
Original    Factor Label
0   Subclinical
1   Clinical
```

Female

```
Original    Factor Label
0   Male
1   Female
```

Treatment

```
Original    Factor Label
0   Control
1   Treatment
```

Also, existing NAs will remain NA.

Verification 1: Check whether the cutoff was applied correctly
Before converting ClinicalDep to a factor, I would run:

```
R
table(
  assign0$ClinicalDep,
  assign0$Depression >= 17,
  useNA = "always"
)
Expected pattern:
```

```
R
      FALSE TRUE
0      xxx    0
1         0   yy
<NA>      0    0
```

However, once you've converted ClinicalDep to a factor, a more useful check is:

```
R
with(assign0,
  table(
    ClinicalDep,
    Depression >= 17,
    useNA = "always"
  ))
You should see:
```

```
R
      FALSE TRUE
Subclinical  xxx    0
```

Clinical 0 yyy
Any non-zero off-diagonal cell indicates an error.

Verification 2: Directly search for mismatches
This should return 0.

```
R
sum(
  assign0$Depression >= 17 &
  assign0$ClinicalDep == "Subclinical",
  na.rm = TRUE
)
This should also return 0.
```

```
R
sum(
  assign0$Depression < 17 &
  assign0$ClinicalDep == "Clinical",
  na.rm = TRUE
)
Verification 3: Check that missing values were preserved
This should return TRUE.
```

```
R
sum(is.na(assign0$Depression)) ==
sum(is.na(assign0$ClinicalDep))
If it returns TRUE, every missing value in Depression remained missing in ClinicalDep.
```

Verification 4: Confirm factor labels
For each factor, inspect frequencies:

```
R
table(assign0$ClinicalDep, useNA = "always")
```

```
R
table(assign0$Female, useNA = "always")
```

```
R
table(assign0$Treatment, useNA = "always")
```

You should see labels rather than numeric values.

Example:

```
R
Subclinical    Clinical    <NA>
      85         32         3
and
```

```
R
Male Female <NA>
  40    78    2
Verification 5: Confirm factor levels exactly
```

```
R
levels(assign0$ClinicalDep)
Expected:
```

```
R
[1] "Subclinical" "Clinical"
```

```
R
levels(assign0$Female)
Expected:
```

```
R
[1] "Male" "Female"
```

```
R
levels(assign0$Treatment)
Expected:
```

```
R
[1] "Control" "Treatment"
```

My preferred verification
If I had to choose only two checks, I would use:

```
R
with(assign0,
  table(
    ClinicalDep,
    Depression >= 17,
    useNA = "always"
  ))
and
```

```
R
sum(is.na(assign0$Depression)) ==
sum(is.na(assign0$ClinicalDep))
Together, they verify both the cutoff rule and the handling of missing values.
```

Check Female and Treatment mappings

Find rows with invalid codes

Create one validation report

Message Copilot

AI-generated content may be incorrect